PRIVACY-ENHANCED SYNTHETIC DATA GENERATION FOR AUTONOMOUS VEHICLE TRAINING

INDIVIDUAL THESIS

Priyadarshi S.A.S.D.

(IT22309556)

Individual component: Privacy Integration

Group ID: R26-CS-010

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) Degree in Information Technology

Specialising in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

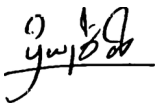
October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

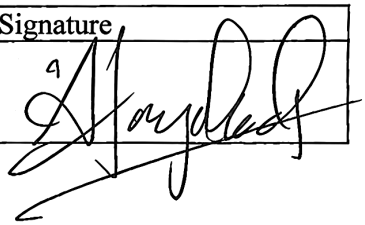
Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Priyadarshi S.A.S.D.
Student ID: IT22309556

Signature:  Date: 03/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Name	Date	Signature
Prof. Anuradha Jayakody	03/10/2026	

Name	Date	Signature
Mr. Dinith Primal	03/10/2026	

ABSTRACT

Autonomous vehicle perception models are trained on large amounts of labeled road data, and this data records people. Driving simulators such as CARLA reduce the need for real footage. However, they are tuned with real driving statistics, and models trained on the generated frames can still reveal their training records. Current platforms give no formal privacy guarantee at either point. This thesis presents the Privacy Integration component of a four-part framework for privacy-enhanced synthetic training data. The aim was to apply differential privacy where real information enters the pipeline and to test the guarantee with real attacks. At calibration, the Laplace mechanism adds noise to seven aggregate driving statistics before they configure the simulator. At training, Opacus DP-SGD with a Rényi differential privacy accountant clips per-sample gradients, adds Gaussian noise and stops training at a hard budget limit. The component then attacks its own models with a shadow-model attack and a loss-threshold attack. The results are stored in HMAC-signed audit reports and a privacy certificate. The evaluation used 10,000 synthetic CARLA frames. A non-private baseline and three budgets (ε = 1, 3 and 8) were each trained with three seeds. Against the baseline, the shadow-model attack reached an AUC of 0.639 and a success rate of 56.7%. Against every private model, the AUC fell to between 0.504 and 0.515, close to random guessing. Paired t-tests showed that this drop was significant ($p \leq 0.0032$), and the cost was 0.16 to 0.21 in test AUC. An independent replication by the Risk and Compliance component agreed within about 2% and found an error in the reported calibration budget, which was corrected. The work shows that a formal ε becomes useful evidence only when it is tested with an attack that is first shown to work.

Keywords: differential privacy, DP-SGD, membership inference, synthetic data, autonomous vehicles

ACKNOWLEDGEMENTS

I would like to thank my supervisor, **Prof. Anuradha Jayakody**, and my co-supervisor, **Mr. Dinith Primal**, for their guidance and feedback throughout this research. Their early advice helped me narrow the privacy work to something I could build and test within the project timeline.

I also thank my team members, Yubitha S. (IT22541284), Apeksha M.K.S.R (IT22110220) and Wijesinghe R.D.P.T. (IT22066916), for their support and cooperation. Having my outputs checked independently by another member shaped many of the design decisions in this thesis. I thank the members of the progress review panels, whose questions helped me explain the size of the privacy effect more clearly. Finally, I thank the developers of PyTorch, Opacus and CARLA, whose open-source tools made this work possible.

TABLE OF CONTENTS

1 INTRODUCTION....1
1.1 Overall project context....1
1.2 Individual component context....2
1.3 Problem statement....3
1.4 Research gap....3
1.5 Aim and objectives....3
1.6 Research questions or hypotheses....4
1.7 Scope limitations and assumptions....4
1.8 Individual contribution....5
1.9 Thesis structure....5
2 LITERATURE REVIEW AND RELATED WORK....6
2.1 Review method and scope....6
2.2 Domain and theoretical foundations....6
2.3 Related research....7
2.4 Existing systems products and techniques....8
2.5 Gap synthesis and conceptual framework....9
3 RESEARCH METHODOLOGY....11
3.1 Research design....11
3.2 Data participants or study materials....12
3.3 Experimental or study procedure....13
3.4 Measures and evaluation metrics....14
3.5 Data analysis....15
3.6 Ethics privacy security and responsible AI....15
3.7 Reproducibility and research artefacts....16
4 PRODUCT AND SOLUTION DESIGN....17
4.1 Requirements....17
4.2 Component architecture within the complete solution....18
4.3 Design alternatives and decisions....19
4.4 Detailed design....20
4.5 Standards and professional practice....22

5 IMPLEMENTATION INTEGRATION AND TESTING.....23
5.1 Development environment and technology stack.....23
5.2 Component implementation.....23
5.3 Integration with the other components.....24
5.4 Verification and software testing.....25
5.5 Deployment and operation.....26
6 RESULTS AND VALIDATION.....27
6.1 Research results.....27
6.2 Product and technical results.....30
6.3 User or stakeholder validation.....30
6.4 Integration results.....31
6.5 Summary of findings.....33
7 DISCUSSION.....34
7.1 Interpretation of findings.....34
7.2 Comparison with literature baselines and existing solutions.....34
7.3 Research contribution.....35
7.4 Product contribution.....35
7.5 Limitations and threats to validity.....36
7.6 Implications.....36
8 COMMERCIALISATION AND SUSTAINABILITY.....38
8.1 Value proposition and target users.....38
8.2 Market and competitive position.....38
8.3 Business or sustainability model.....39
8.4 Scalability intellectual property and compliance.....39
8.5 Contribution to the Sustainable Development Goals.....40
9 CONCLUSION AND FUTURE DIRECTIONS.....41
9.1 Conclusion.....41
9.2 Achievement of objectives.....41
9.3 Individual research and product contributions.....42
9.4 Future directions.....42
REFERENCES.....44

GLOSSARY.....46
APPENDICES.....47
Appendix A Extended results.....47
Appendix B Academic integrity and AI use disclosure.....47

LIST OF FIGURES

Figure 1.1: Overall system overview of the R26-CS-010 framework.....2

Figure 2.1: Conceptual framework of the Privacy Integration component.....10

Figure 3.1: Research design and experimental procedure of the component.....11

Figure 4.1: Group pipeline architecture and the two points where Privacy Integration acts..... 18

Figure 4.2: Internal data flow of the Privacy Integration component.....20

Figure 6.1: Test AUC and attack AUC by configuration (mean and one standard deviation over three seeds)..... 27

Figure 6.2: Attack AUC of the shadow-model and loss-threshold attacks.....29

Figure 6.3: Target and final logged ε of the three private configurations (independent audit by Risk and Compliance).....32

Figure 6.4: Attack AUC reported by Privacy Integration and found by the independent replication.....32

LIST OF TABLES

Table 2.1: Existing systems and techniques compared with the needs of the component....8

Table 2.2: Research gaps addressed by the Privacy Integration component....9

Table 3.1: Dataset after label derivation and splitting....12

Table 3.2: Experimental configurations....13

Table 3.3: Measures and evaluation metrics....14

Table 4.1: Functional requirements of the Privacy Integration component....17

Table 4.2: Design alternatives and decisions....19

Table 4.3: Calibration queries, sensitivities and Laplace noise scale at ε = 1.8 per query....20

Table 4.4: Presence classifier architecture (input 3 × 336 × 448)....21

Table 5.1: Development environment and technology stack....23

Table 5.2: Files exchanged by the Privacy Integration component....24

Table 5.3: Verification of the functional requirements....25

Table 6.1: Privacy and utility by configuration (mean ± standard deviation over three seeds)....27

Table 6.2: Paired t-tests of each private configuration against the baseline (three seeds)....28

Table 6.3: Loss-threshold attack (mean ± standard deviation over three seeds; random guessing gives an AUC of 0.5 and a TPR equal to the FPR)....28

Table 6.4: Privatized calibration statistics sent to Dataset Generation....29

Table 6.5: Independent verification of the component by Risk and Compliance....31

Table 9.1: Achievement of objectives....41

Table A.1: Privacy record of each private configuration....47

Table A.2: Reconstruction check on the calibration channel....47

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AUC	Area Under the ROC Curve
AV	Autonomous Vehicle
CARLA	Car Learning to Act (open-source driving simulator)
CBC	Cipher Block Chaining
CNN	Convolutional Neural Network
DP	Differential Privacy
DP-SGD	Differentially Private Stochastic Gradient Descent
FPR	False-Positive Rate
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
HMAC	Hash-based Message Authentication Code
JSON	JavaScript Object Notation
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute (dataset)
mAP	Mean Average Precision
MIA	Membership Inference Attack
NIST	National Institute of Standards and Technology
RDP	Rényi Differential Privacy
ROC	Receiver Operating Characteristic
RQ	Research Question
SD	Standard Deviation
SDG	Sustainable Development Goal
SGD	Stochastic Gradient Descent
SHA	Secure Hash Algorithm
TPR	True-Positive Rate
YOLO	You Only Look Once (object detector)

1 INTRODUCTION

1.1 Overall project context

Autonomous vehicles use machine learning to detect objects, predict how other road users will move and plan how to drive. These models need large amounts of labeled sensor data. Public datasets such as KITTI [1] and the Waymo Open Dataset [2] were built by driving sensor-equipped cars on public roads. The recordings were then labeled by hand. This data contains personal information. Cameras record faces and license plates, and repeated routes can show where people live and work. Under the General Data Protection Regulation (GDPR), recordings of identifiable people are personal data. Article 25 requires data protection by design and by default [3].

One response is to generate training data in a simulator such as CARLA [4]. A simulator removes the need to record real people, but it does not remove privacy risk on its own. It must be calibrated with real observations, such as traffic density, weather and time of day, so real information still enters the pipeline. Models trained on the generated data can also reveal which records were in their training set [5], [6]. Stadler et al. [7] showed that synthetic data does not protect its source records unless a formal privacy mechanism is used.

Project R26-CS-010 addresses these risks with a privacy-enhanced synthetic data generation framework. The framework has four components. Privacy Integration (IT22309556) applies and tests differential privacy where real information enters the pipeline. Dataset Generation (IT22541284) captures CARLA frames and protects every file with SHA-256 hashing, AES-256 encryption and role-based access control. Utility Evaluation (IT22110220) checks the data for tampering and measures how useful it is for training a detector. Risk and Compliance (IT22066916) verifies the privacy controls with independent attacks and maps the evidence to GDPR and NIST requirements.

1.2 Individual component context

This thesis covers the Privacy Integration component. Real information can enter the pipeline at two points, and the component is responsible for both. The first point is calibration, where real aggregate statistics configure the simulator. The second point is model training, where a model may memorize the records it was trained on. The component applies differential privacy at each point and then tests whether the protection holds against an attacker.

The component sits between the data owner and the rest of the framework. It receives aggregate statistics on the data owner's machine and sends only privatized values to Dataset Generation. Later in the flow, it receives the encrypted frames from Dataset Generation and the clean labels from Utility Evaluation. It trains models on them, attacks those models and sends signed evidence to Risk and Compliance. Its outputs are therefore inputs to two other components, and its claims are checked by a third. Figure 1.1 shows how the four components connect.

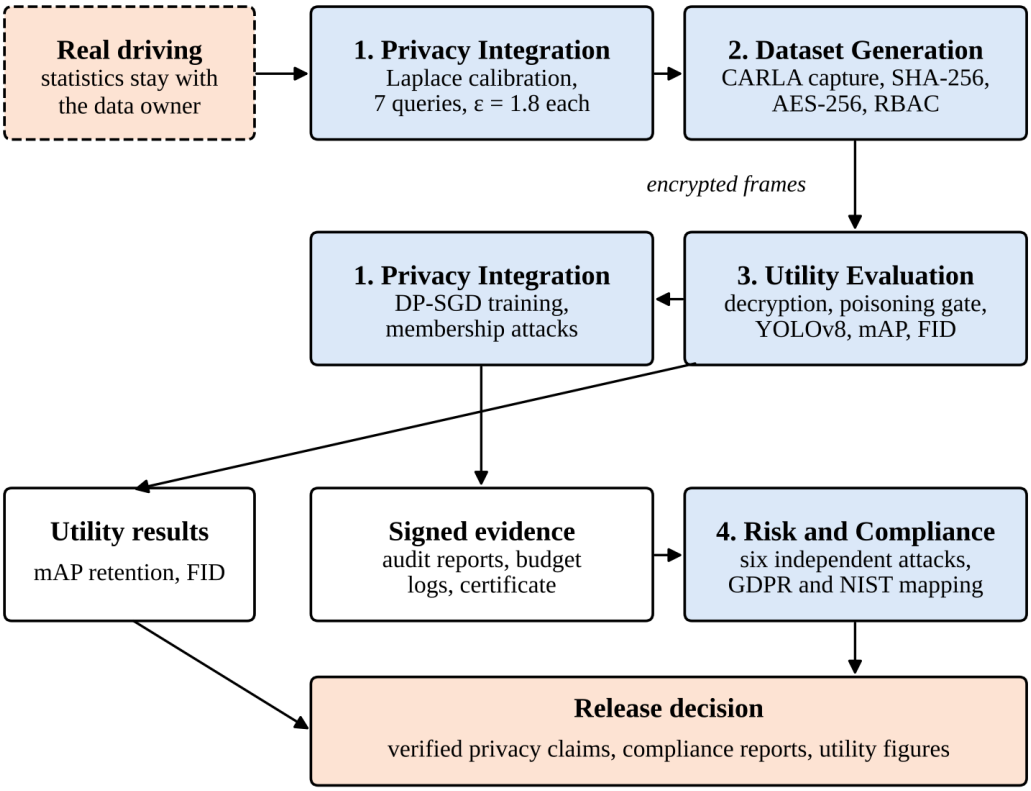


Figure 1.1: Overall system overview of the R26-CS-010 framework

1.3 Problem statement

Autonomous vehicle simulators are configured from real driving statistics, but current platforms compute and pass on these statistics without any privacy protection. They also give no formal guarantee for the models trained on their output. Differentially private training has been applied to generative models, but mainly on small benchmarks such as MNIST [8], [9]. Privacy budgets are also often chosen for model accuracy and are rarely checked against measured leakage [10], [11]. As a result, developers cannot show, in a way others can check, that their synthetic driving data does not expose the people in the real data.

1.4 Research gap

Earlier work leaves four gaps that matter for this component. First, simulator calibration is treated as a configuration task, so the real statistics behind it receive no protection [4]. Second, differentially private training has been tested mainly on benchmark datasets and not on driving imagery [12], [8]. Third, a formal budget ε is only a bound on paper. Few studies test it with a real attack against both private and non-private models [10], [11]. Fourth, privacy claims are rarely packaged so that a separate party can verify them. Section 2.5 explains these gaps in more detail.

1.5 Aim and objectives

The aim of this research is to integrate ε-differential privacy into the synthetic data pipeline so that its privacy can be measured and verified. The project proposal set three measurable targets. The cumulative budget of model training should not exceed $\varepsilon = 10$. The success rate of a membership inference attack against the private models should stay at or below 55%, where random guessing gives 50%. Perception accuracy should stay within 15% of a model trained on real data. The specific objectives are:

1. To select a differentially private training library that supports per-sample gradients and Rényi differential privacy (RDP) accounting.
2. To protect the calibration step with the Laplace mechanism, keeping the calibration budget below $\varepsilon = 2$.
3. To apply DP-SGD during training and stop training automatically if the budget limit is reached.

4. To attack every trained model with a shadow-model membership inference attack, using a non-private baseline as a positive control.
5. To measure the trade-off between privacy and utility over three budgets and several random seeds.
6. To produce signed evidence that Risk and Compliance can verify independently.

1.6 Research questions or hypotheses

The main research question of this component is:

RQ: Can differential privacy at calibration and training stop membership inference on synthetic driving data at a cost that can be measured?

The question was split into three sub-questions:

- **RQ1:** Can the seven calibration statistics be privatized before they reach the simulator, without changing how the simulator is used?
- **RQ2:** Does DP-SGD reduce the success of membership inference to near random guessing, when the same attack succeeds against a non-private model?
- **RQ3:** What does this protection cost in model utility, and does the cost change with the budget ε?

The working hypothesis was that the attack would succeed against the non-private baseline. Against every private model, it would fall to near 50%, with a measurable drop in test AUC.

1.7 Scope limitations and assumptions

The component covers two privacy points: the statistics that calibrate CARLA and the models trained on the generated frames. It does not train a generative model on real records, because CARLA renders the frames directly from the privatized settings. The Laplace step is therefore where real information receives its formal guarantee. Applying DP-SGD to a classifier trained on the synthetic frames was a scope decision. It made it possible to test how ε relates to resistance against a real attack.

The work makes four assumptions. The source statistics were reference values that Dataset Generation measured from CARLA runs. They stood in for statistics from a

real dataset such as the Waymo Open Dataset. Privacy was measured on a binary task (whether a frame contains a pedestrian or a cyclist), because a simple task gives a clear membership signal. The attacker was assumed to have query access to the model and data from the same distribution. Encryption, access control and file integrity were handled by the other components.

1.8 Individual contribution

The author designed and built the whole Privacy Integration component. The main contributions are:

- A calibration module that applies the Laplace mechanism to seven driving statistics and writes a public file and a private audit log.
- A DP training wrapper based on Opacus that logs ε after every step and stops training at a hard limit.
- A small convolutional classifier that trains under DP-SGD, together with a label rule that removed label noise from the dataset.
- A shadow-model membership inference attack with a non-private positive control, and a second, loss-threshold attack.
- HMAC-signed audit reports and a combined privacy certificate with paired t-tests across seeds.
- The JSON contracts and files that connect the component to the other three components, and 107 unit tests.

1.9 Thesis structure

Chapter 2 reviews related work and builds the conceptual framework. Chapter 3 explains the research design, data, procedure and metrics. Chapter 4 presents the requirements and the design of the component. Chapter 5 covers its implementation, integration and testing. Chapter 6 reports the results, and Chapter 7 discusses them. Chapter 8 considers commercialization and sustainability, and Chapter 9 concludes the thesis.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review covered four topics that the component depends on. These were the theory of differential privacy, its use in deep learning, the privacy of synthetic data and simulators, and attacks that measure leakage. Most sources are peer-reviewed papers from security and machine learning venues. Examples are the IEEE Symposium on Security and Privacy, the USENIX Security Symposium, ACM CCS and CVPR. Preprints were used where they describe a tool that the component uses, such as Opacus, or recent work on autonomous vehicle privacy. Regulations and standards, such as the GDPR and the NIST Privacy Framework, were used to define what the evidence should support. Work was included if it defined a mechanism the component uses, tested privacy on images or synthetic data, or measured privacy with an attack.

2.2 Domain and theoretical foundations

Dwork et al. [13], [14] defined differential privacy as a property of a randomized computation. A mechanism M satisfies ε-differential privacy if, for any two datasets D and D′ that differ in one record, and for any set of outputs S:

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \cdot \Pr[M(D') \in S]$$

A smaller ε gives stronger protection but needs more noise. The relaxed form, (ε, δ)-differential privacy, allows this limit to fail with a small probability δ and is used for deep learning [15]. The Laplace mechanism answers a numerical query f by adding Laplace noise with scale $\Delta f / \varepsilon$. Here, the sensitivity Δf is the largest change that one record can cause [13]. Two properties matter for this study. Post-processing of a private output does not weaken the guarantee. Under sequential composition, k mechanisms run on the same records with budgets ε_1 to ε_k together satisfy $(\varepsilon_1 + ... + \varepsilon_k)$-differential privacy [15].

Abadi et al. [12] introduced differentially private stochastic gradient descent (DP-SGD). The gradient of each sample is clipped to a maximum L2 norm C, and Gaussian noise with standard deviation σC is added to the sum before averaging over the batch of size L:

$$\tilde{g} = (1/L) \cdot (\Sigma_i \, \mathrm{clip}(g_i, C) + N(0, \sigma^2 C^2 I))$$

Clipping limits how far one record can move the weights, and the noise hides the remaining effect. Mironov [16] introduced Rényi differential privacy (RDP). It generalizes the moments accountant of Abadi et al. and composes more tightly than standard (ε, δ) composition over many training steps.

Autonomous vehicle data adds its own privacy risks. Xie et al. [17] surveyed privacy risks in autonomous vehicles and listed protection methods ranging from anonymization to private learning. The GDPR treats recordings of identifiable people as personal data and asks for data minimization and accountability [3]. Ganev [18] discussed when synthetic data can count as anonymous under such rules, and argued that formal privacy and attack testing matter for this question.

2.3 Related research

Private synthetic data. CARLA [4] is an open-source urban driving simulator that produces camera, depth and segmentation data with automatic labels. Its traffic, pedestrians and weather can be set to match a real city. However, the CARLA paper does not discuss how the real statistics behind these settings should be protected. For learned generators, DP-GAN [8] adds Gaussian noise to the gradients of the discriminator in a generative adversarial network, and DP-CGAN [9] adds label conditioning. These works were tested on MNIST and medical records, not on driving data. Stadler et al. [7] found that synthetic data without a formal mechanism did not reliably protect the source records. Private generators protected them better, but at a cost in utility. Protection must therefore be placed where real records affect the output.

Membership inference. Shokri et al. [5] introduced the shadow-model attack. The attacker trains shadow models that copy the target and records how they respond to their own training records and to other records. An attack classifier is then trained on these responses. Yeom et al. [19] linked membership leakage to overfitting and showed

that the per-sample loss is a strong membership signal. Carlini et al. [6] showed that average measures such as the area under the ROC curve (AUC) can hide attacks that succeed on a few records. They recommended reporting the true-positive rate at low false-positive rates.

Auditing DP in practice. Jayaraman and Evans [10] found that the leakage measured at a given ε varies widely between settings. Nasr et al. [11] used attacks to audit DP-SGD. They showed that a worst-case attacker can reach the formal bound, while realistic attackers fall well short of it. These studies agree on two points. A formal ε should be tested with an attack. A non-private model must also be attacked, to show that the attack works at all.

2.4 Existing systems products and techniques

Several tools and practices already address parts of the problem. Table 2.1 compares them with the needs of this component.

Table 2.1: Existing systems and techniques compared with the needs of the component

System or technique	What it provides	Limitation for this work
Face and plate blurring	Removes visible identifiers from images	Does not protect against membership inference at the model level
CARLA, AirSim and similar simulators	Labeled synthetic sensor data	No formal privacy guarantee; the calibration step is not protected [4]
DP-GAN and DP-CGAN	Private generative models [8], [9]	Tested on MNIST and medical data, not on driving data
Opacus	DP-SGD with per-sample gradients and an RDP accountant for PyTorch [20]	A training library only; it does not test the trained model with an attack
ML Privacy Meter	Membership inference audits of trained models [21]	Measures leakage but does not apply privacy or link results to a budget log

Opacus is the closest match for the training step. It implements DP-SGD and an RDP accountant for PyTorch and supports the Poisson sampling of batches that its accounting assumes [20]. It does not allow batch normalization, which mixes information across a batch, so private models commonly use group normalization [22]

instead. Auditing tools such as ML Privacy Meter [21] measure leakage after training. They do not apply the protection, and they do not link the leakage to a signed record of the budget.

2.5 Gap synthesis and conceptual framework

Table 2.2 summarizes the gaps that this component addresses and the approach taken for each.

Table 2.2: Research gaps addressed by the Privacy Integration component

Gap	Limitation of existing work	Approach in this component
Calibration privacy	Simulator statistics are computed from real data with no protection [4]	Laplace mechanism on every statistic before it reaches the simulator
DP on driving data	DP training was tested on benchmarks such as MNIST [12], [8]	DP-SGD on synthetic CARLA camera frames
Formal bound and attack	Budgets are chosen for accuracy and rarely checked with attacks [10], [11]	Shadow-model and loss-threshold attacks on every model, with a non-private control
Verification	Privacy claims are rarely prepared for third-party checks	Signed audit reports, budget logs and a certificate checked by another component

These gaps lead to the conceptual framework in Figure 2.1. Real information enters at two points, and each point has its own mechanism. The Laplace mechanism protects the calibration statistics, and DP-SGD protects the trained models. Each mechanism produces evidence: a budget record, attack results against private and non-private models, and a signature over the report. A separate party then checks this evidence. Privacy is claimed only when all three kinds of evidence agree.

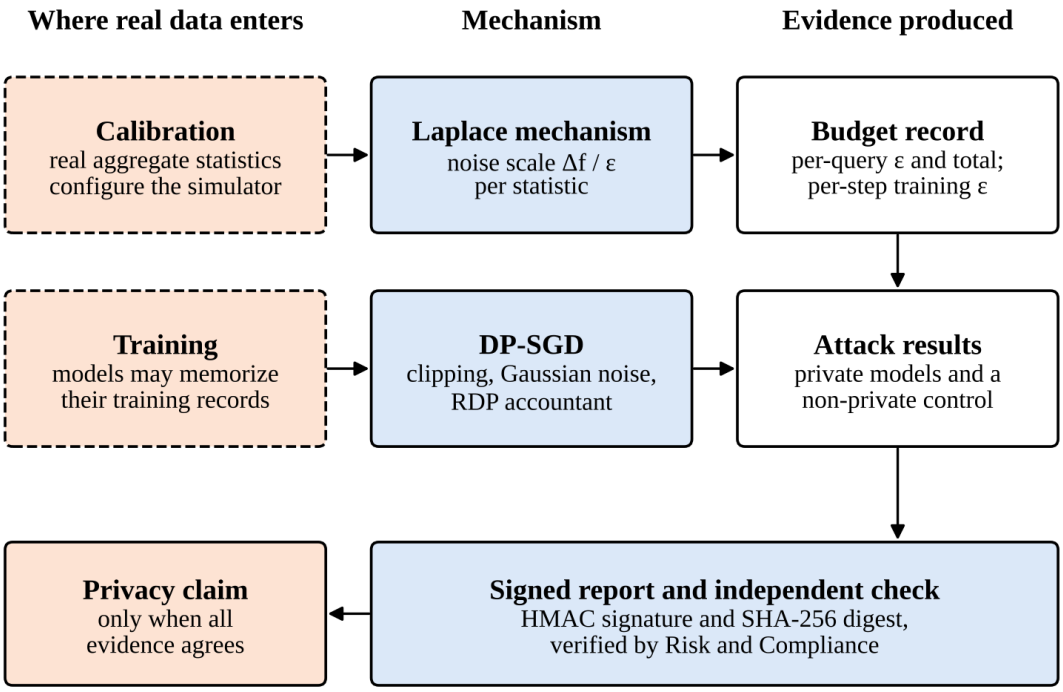


Figure 2.1: Conceptual framework of the Privacy Integration component

3 RESEARCH METHODOLOGY

3.1 Research design

The research followed a quantitative, experimental design inside a build-and-evaluate cycle. The component was first built as a working artifact. It was then tested with controlled experiments in which the privacy setting was the only factor that changed. The design had four parts:

- **Positive control.** A non-private baseline was trained and attacked in the same way as the private models. If the attack had failed against it, a result near 50% for the private models would not have proved privacy.
- **Manipulated factor.** The privacy budget took four levels: no DP, and DP-SGD with a target ε of 1, 3 and 8.
- **Repetition.** Each level was trained with three random seeds, so that differences could be tested for significance.
- **Independent check.** Risk and Compliance re-ran the key checks with its own code and compared the results.

Figure 3.1 shows the research process from the problem to the verified results.

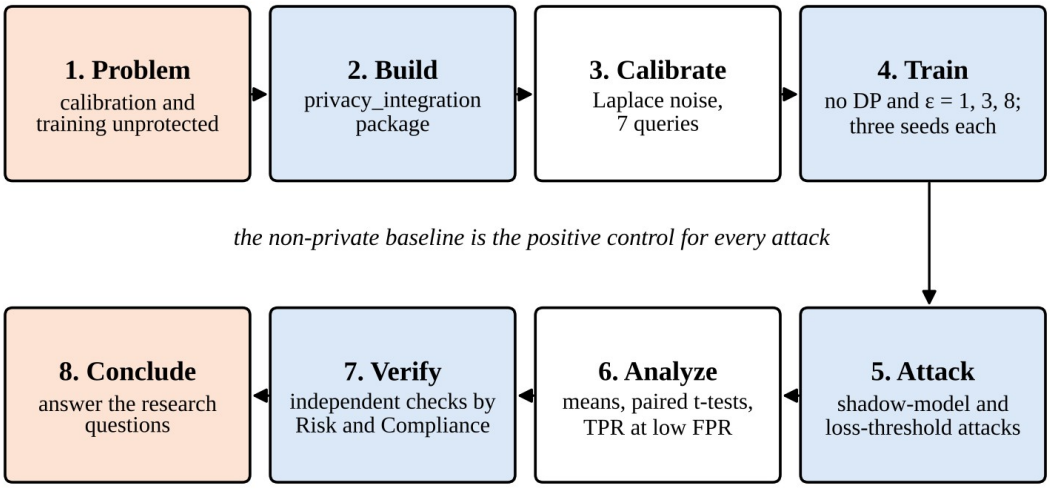


Figure 3.1: Research design and experimental procedure of the component

3.2 Data participants or study materials

No human participants took part in this study. The material was synthetic data and aggregate statistics.

Calibration statistics. The simulator needs seven aggregate statistics. These are vehicle and pedestrian density per kilometer, average vehicle and pedestrian speed, intersections per kilometer, and the weather and time-of-day distributions. The design takes these from a real dataset such as the Waymo Open Dataset [2]. In this study, reference values measured by Dataset Generation from CARLA runs were used in their place.

Training frames. The training data was the five-town CARLA dataset of 10,000 frames, captured at 800 × 600 pixels by Dataset Generation. Every image was encrypted with AES-256 in CBC mode and listed in a SHA-256 manifest. The component decrypted each file and checked it against the manifest before use. The labels came from the poisoning gate of Utility Evaluation.

Labels. Privacy was measured on a binary question: does the frame contain a pedestrian or a cyclist? A frame with neither was negative. A frame was positive when its largest pedestrian or cyclist covered at least 1,024 pixels (about 32 × 32). Frames with only smaller targets were excluded, because labeling them negative would have been wrong. This follows the idea behind the KITTI benchmark, which leaves out objects too small to label reliably [1]. Table 3.1 summarizes the dataset after labeling and splitting.

Table 3.1: Dataset after label derivation and splitting

Item	Frames	Share
Positive (pedestrian or cyclist large enough to see)	796	8.0%
Negative (no pedestrian or cyclist)	7,052	70.5%
Excluded (only small targets)	2,152	21.5%
Training split (stratified)	5,493	70% of usable
Validation split	1,176	15% of usable
Test split	1,179	15% of usable

The 7,848 usable frames were split 70/15/15 with stratification. Images were resized to 448 × 336, which keeps the 4:3 aspect ratio. The training split was balanced by duplicating positive frames.

3.3 Experimental or study procedure

The procedure had five steps.

1. **Private calibration.** Each of the seven statistics was answered through the Laplace mechanism with ε = 1.8 per query. The privatized file was passed to Dataset Generation, and the noise log was passed only to Risk and Compliance.
2. **Data preparation.** The encrypted frames were decrypted, checked against the manifest, labeled with the rule in Section 3.2 and split with a fixed seed.
3. **Training.** A non-private baseline and three private configurations were each trained with seeds 42, 43 and 44, giving twelve models.
4. **Attack.** Every model was attacked with the shadow-model attack and then with the loss-threshold attack.
5. **Evidence.** Every run produced a signed audit report, and a certificate combined all runs and the significance tests.

Table 3.2 lists the four configurations. Opacus took the target ε, $\delta = 10^{-5}$, the number of epochs and the sampling rate as inputs and computed the noise multiplier σ from them.

Table 3.2: Experimental configurations

Configuration	Target ε	Mechanism	Seeds
No DP (baseline)	none	Adam, 15 epochs	42, 43, 44
Config A	1.0	DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, σ = 0.977	42, 43, 44
Config B	3.0	DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, σ = 0.665	42, 43, 44
Config C	8.0	DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, σ = 0.507	42, 43, 44

Per-sample gradients were clipped to C = 1.0. The private models used SGD with momentum, a learning rate of 10^{-3} and a batch size of 32. The baseline used the same model and data, but with Adam, a learning rate of 10^{-4} and 15 epochs. The RDP

accountant was read after every step, and its cumulative ε was written to a budget log. If ε had reached the hard limit of 10, training would have stopped with an error.

For the shadow-model attack, a shadow model was trained on half of the data with the same privacy setting as the target [5]. For each of its members and non-members, two features were recorded: the confidence and the per-sample loss [19]. A small neural network learned to separate members from non-members using these features and was then applied to the target. The members were 1,179 training images of the target, and the non-members were its 1,179 test images. With equal set sizes, random guessing scores exactly 50%. The loss-threshold attack needed no shadow model. It treated a lower loss on the target model as a sign of membership and used the same members and non-members.

3.4 Measures and evaluation metrics

Table 3.3 lists the measures used in the evaluation.

Table 3.3: Measures and evaluation metrics

Measure	Definition	Purpose
Achieved ε	Final value of the RDP accountant at $\delta = 10^{-5}$	Formal privacy cost of training
Attack success rate	Share of members and non-members classified correctly	Main proposal target (≤ 55%)
Attack AUC	Area under the ROC curve of the attack	Leakage over all thresholds (0.5 = chance)
TPR at low FPR	True-positive rate at 1% and 5% false-positive rate [6]	Leakage on the most exposed records
Loss gap	Mean loss of non-members minus mean loss of members	Size of the overfitting signal
Test AUC	AUC of the classifier on the test split	Utility of the trained model

Test AUC was used for utility instead of accuracy. The test split keeps the natural positive rate of about 10%. A model that always predicts "no pedestrian" would therefore have a high accuracy and still be useless.

3.5 Data analysis

Each measure was reported as the mean and standard deviation over the three seeds. ROC curves and AUC values were computed with scikit-learn. Each private configuration was compared with the baseline trained with the same seed. A paired t-test was used on the drop in test AUC and on the drop in attack AUC. A result was taken as significant at the 0.05 level. With three seeds, the tests can separate a private model from the baseline, but they have little power to separate the three budgets from each other. This limit is discussed in Section 7.5.

For the calibration step, the analysis checked two things. The simulator configuration of Town01 was compared field by field with the privatized file. The total calibration budget was computed under sequential composition, because all seven queries read the same records.

3.6 Ethics privacy security and responsible AI

All images in this work were generated inside the CARLA simulator, so no real person, vehicle or place was recorded. The calibration source values were reference statistics from CARLA runs and contain no records of real people. Even so, the component treated them as if they were real. The calibration audit log records the noise drawn for each query, so subtracting it from a published value gives back the source value. For this reason, the audit log was shared only with Risk and Compliance and was never published.

The AES dataset key and the HMAC signing key were kept separate. Both were loaded from a local .env file that is excluded from version control. Before the full experiment, a code review found both keys written into source files of a repository that was going to be published. Both keys were replaced, and a regression test now fails if an old key appears in the source again.

Every privacy claim in this thesis is tied to an attack result against a positive control, and known weaknesses are recorded in the signed certificate. The component does not claim legal compliance. It produces evidence that a data controller can use when making that decision.

3.7 Reproducibility and research artefacts

Every source of randomness was seeded, and repeating a run with a fixed seed gave the same loss and the same test AUC. All settings were kept in one file, config.yaml, and conflicting values were rejected when the package loaded. Results were saved in a separate folder for each seed, so a new run never overwrote an earlier one. The software versions were fixed at Python 3.11, PyTorch 2.5.1 and Opacus 1.4.1.

The main research artefacts are the privacy_integration package, the twelve trained models, the per-step budget logs, the signed audit reports and the privacy certificate. Each report is stored with its SHA-256 digest, so a later change to any artefact can be detected.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from the project proposal, the needs of the other three components and the stakeholders named in the project proposal. These stakeholders were autonomous vehicle developers, who need training data with an auditable privacy record. Road users, whose data should not be exposed, were also included. So were data protection regulators, who need evidence they can check, and researchers who want to share datasets safely. Table 4.1 lists the functional requirements.

Table 4.1: Functional requirements of the Privacy Integration component

ID	Requirement	Source
FR-01	Every private training run stays within its target ε	Proposal target
FR-02	Per-sample gradients are clipped and noise is added at every step	Definition of DP-SGD [12]
FR-03	Training stops automatically if ε reaches the hard limit of 10	Proposal target
FR-04	Noise is added to the statistics before they reach the simulator	Calibration gap
FR-05	Every run produces a signed report that fails verification if changed	Needs of Risk and Compliance
FR-06	The cumulative ε is logged after every training step	Needs of Risk and Compliance

The non-functional requirements were as follows:

- **Security.** Keys are never written in the source code, and files that could reveal source values are never published.
- **Verifiability.** Another party can check every claim from the files alone, without rerunning the training.
- **Reproducibility.** A run repeated with the same seed gives the same result.
- **Interoperability.** Every file exchanged with another component follows a JSON Schema contract.
- **Resource limits.** Training and attacks run on a laptop GPU with 4 GB of memory.

4.2 Component architecture within the complete solution

Figure 4.1 shows where the component sits in the group pipeline. It works at two points. Before the simulator, it receives aggregate statistics on the data owner's machine and sends only privatized values to Dataset Generation. After the simulator, it trains models on the synthetic frames with DP-SGD, attacks these models and sends signed evidence to Risk and Compliance. The poisoning gate of Utility Evaluation supplies the clean labels used for training.

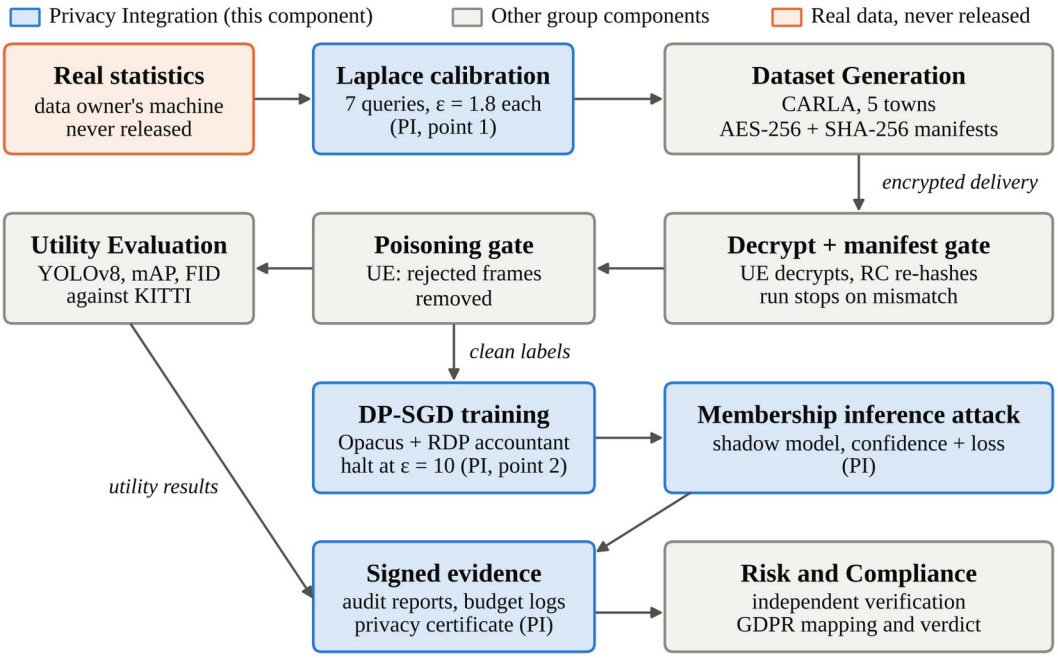


Figure 4.1: Group pipeline architecture and the two points where Privacy Integration acts

This position gives the component a clear boundary. The file calibration_stats.json is the only path from the source statistics to the simulator. The signed reports are the only path from the training runs to the auditor. Both paths can therefore be checked in one place.

4.3 Design alternatives and decisions

Table 4.2 lists the main design choices, the alternatives that were considered and the reason for each choice.

Table 4.2: Design alternatives and decisions

Decision	Alternative	Reason for the choice
Privatize the calibration statistics	Train a private generator on real records	CARLA renders frames from its settings, so the statistics are the only point where real data enters the generator
Laplace mechanism for calibration	Gaussian mechanism	Gives pure ε-differential privacy for a small number of numeric queries [13]
Opacus for DP-SGD	TensorFlow Privacy	Computes per-sample gradients without model changes, provides RDP accounting and works with the PyTorch code of the group [20]
RDP accountant	Basic sequential composition	Composes more tightly over thousands of training steps [16]
Group normalization	Batch normalization	Opacus does not allow batch normalization, which mixes samples across a batch [22]
SGD with momentum for private runs	Adam	Adam was unstable under DP-SGD in the early runs
Duplicating positive frames	Weighted sampler or class weights	Opacus replaces a weighted sampler, and class weights did not work well with per-sample clipping
Shadow-model and loss-threshold attacks	A single attack	Two different attacks reduce the chance that one weak attack hides leakage [5], [19]
HMAC-SHA256 signatures	Public-key signatures	Simple to verify with a shared key; public-key signatures are noted as future work

The first decision shaped the rest of the design. The final pipeline has no generative model trained on real records, because CARLA renders the frames directly from the privatized settings. DP-SGD was therefore applied to a classifier trained on the synthetic frames. This choice made it possible to test how ε relates to resistance against a real attack. That link was one of the gaps in Section 2.5.

4.4 Detailed design

The component is an installable Python package called privacy_integration. It has six modules, and Figure 4.2 shows how data moves between them.

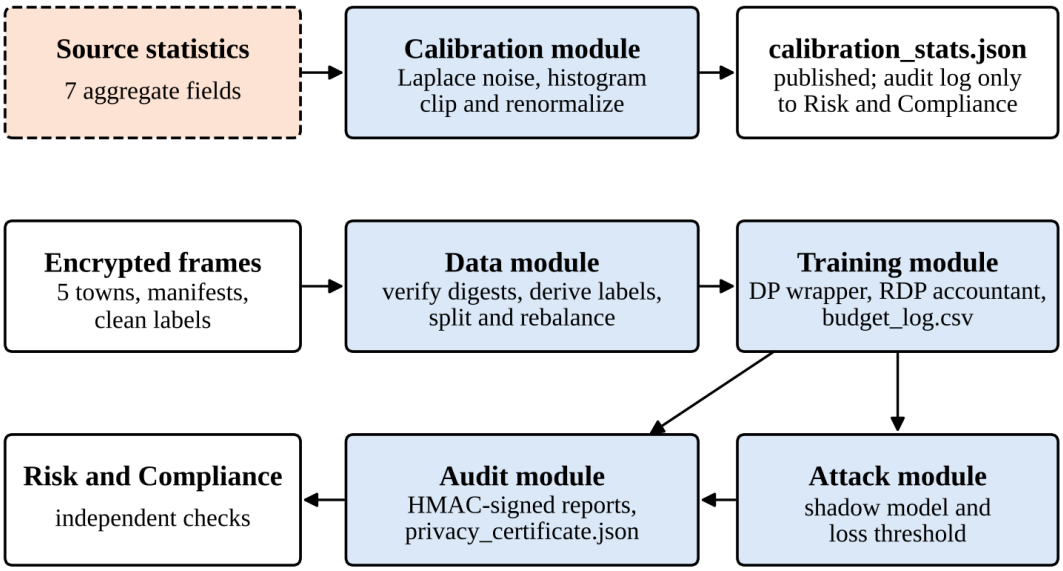


Figure 4.2: Internal data flow of the Privacy Integration component

Calibration module. Each statistic is one query answered through the Laplace mechanism with ε = 1.8, which is below the configured per-query limit of 2.0. The sensitivity is 2.0 for vehicle speed and 1.0 for the other queries. For the two distributions, noise is added to every bin, negative bins are set to zero and the bins are scaled to sum to one. This is post-processing, so it does not weaken the guarantee. Table 4.3 lists the queries.

Table 4.3: Calibration queries, sensitivities and Laplace noise scale at ε = 1.8 per query

Query	Type	Sensitivity Δf	Noise scale b
Vehicle density per km	Scalar	1.0	0.556
Pedestrian density per km	Scalar	1.0	0.556
Average vehicle speed (km/h)	Scalar	2.0	1.111
Average pedestrian speed (km/h)	Scalar	1.0	0.556
Intersections per km	Scalar	1.0	0.556
Weather distribution (4 bins)	Histogram	1.0	0.556 per bin

Query	Type	Sensitivity Δf	Noise scale b
Time-of-day distribution (4 bins)	Histogram	1.0	0.556 per bin

All seven queries read the same records, so their costs add up under sequential composition to 7 × 1.8 = 12.6. The module writes two files. The file calibration_stats.json holds the privatized statistics. The file calibration_audit_log.json records the noise drawn for each query and goes only to Risk and Compliance.

Data module. This module decrypts each frame, checks it against the SHA-256 manifest, applies the label rule and creates the stratified splits.

Model module. The classifier is a small convolutional network, listed in Table 4.4. Group normalization replaces batch normalization. Max pooling and limited downsampling help small pedestrians reach the output.

Table 4.4: Presence classifier architecture (input 3 × 336 × 448)

Stage	Operation	Output channels
Block 1	3 × 3 convolution, stride 2, GroupNorm, ReLU	32
Block 2	3 × 3 convolution, stride 2, GroupNorm, ReLU	64
Block 3	3 × 3 convolution, stride 2, GroupNorm, ReLU	128
Block 4	3 × 3 convolution, stride 1, GroupNorm, ReLU	128
Pooling	Adaptive max pooling to 1 × 1	128
Classifier	Linear 128 to 64, ReLU, Linear 64 to 1	1 (logit)

Training module. This module holds the DP training wrapper, the budget accountant and the non-private baseline.

Attack module. This module trains the shadow models, builds the attack classifier and runs the loss-threshold attack.

Audit module. This module signs the reports and builds the privacy certificate. After every private run, it writes an audit report with the final ε, C, σ, δ, the limit and the

number of steps. The report is signed with HMAC-SHA256, and its SHA-256 digest is stored next to it. For the baseline, a signed declaration states that no privacy mechanism was applied.

4.5 Standards and professional practice

The design follows the standards and practices listed below.

- **Data protection by design.** Privacy is applied where real data enters the pipeline, as GDPR Article 25 requires [3]. Only privatized statistics leave the data owner, which supports the data minimization principle of Article 5(1)(c).
- **Accountability.** Signed reports and budget logs support the accountability duty of GDPR Article 5(2) [3].
- **Risk frameworks.** The evidence supports the Control and Protect functions of the NIST Privacy Framework [23]. It also supports the privacy and security goals of the NIST AI Risk Management Framework [24].
- **Cryptographic standards.** Digests use SHA-256 as defined in FIPS 180-4 [25].
- **Engineering practice.** Secrets were kept out of version control. Code was reviewed before the full experiment, and unit tests ran on every push through continuous integration.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

All training and attack runs used a laptop with an NVIDIA RTX 3050 Ti GPU (4 GB) running Windows 11. Table 5.1 lists the main tools and their purpose.

Table 5.1: Development environment and technology stack

Tool	Purpose
Python 3.11	Main programming language
PyTorch 2.5.1	Building and training the convolutional classifier
Opacus 1.4.1	DP-SGD with per-sample gradients and an RDP accountant
NumPy and SciPy	Laplace noise, statistics and paired t-tests
scikit-learn	ROC curves and AUC for the attacks
PyCryptodome	Decrypting the dataset received from Dataset Generation
Python hmac and hashlib	HMAC-SHA256 signatures and SHA-256 digests
JSON Schema	Checking every file exchanged with other components
pytest and GitHub Actions	107 unit tests and continuous integration

5.2 Component implementation

Each module of the package has a command-line entry point, so every stage can be run and checked on its own. All settings are read from config.yaml when the package loads.

Training wrapper. The DPTrainingWrapper class attaches an Opacus PrivacyEngine to the model, optimizer and data loader. It calls make_private_with_epsilon to find the noise multiplier σ for the target ε. After every step it reads the accountant and adds a row to the budget log. If the cumulative ε reaches the limit, it raises BudgetExceededError and stops the loop. Opacus secure mode was turned off. Secure mode needs the torchcsprng library, which is no longer actively maintained and requires PyTorch 1.8.1. It cannot be used with PyTorch 2.5 on any operating system, so the certificate lists this as a limitation.

Audit and certificate. The CertificateSigner class signs each report. The key is read from the project .env file, and verification uses a constant-time comparison. The

build_certificate script reads the results of every run, computes the paired t-tests and writes a signed privacy_certificate.json.

Implementation challenges. In the first runs on real frames, both the private and the non-private models predicted the majority class for every input. A search over learning rates, optimizers, batch sizes and imbalance methods gave three findings. Adam was unstable under DP-SGD, so private training was switched to SGD with momentum. Loss-based class weighting did not work well with per-sample clipping. Opacus also replaces a weighted sampler, so balancing had to be done by duplicating frames.

None of these changes fixed the problem. A label check then found that in 62.3% of positive frames, the largest pedestrian was smaller than about 8 × 8 pixels after resizing. Most positives were therefore label noise. The frames were also being stretched into a 256 × 256 square. After the aspect ratio was kept and frames with only small targets were excluded, the baseline test AUC rose from about 0.50 to 0.84. The earlier failure came from data preparation and not from differential privacy.

5.3 Integration with the other components

A pipeline runner connects the four components in order. Before any training starts, Risk and Compliance re-hashes the decrypted dataset, and the poisoning gate of Utility Evaluation must produce the clean labels. Every file passed between components is checked against a JSON Schema contract, so a missing or wrongly typed field stops the run at that stage. Table 5.2 lists the files that the component exchanges.

Table 5.2: Files exchanged by the Privacy Integration component

File	Direction	Purpose
calibration_stats.json	To Dataset Generation and Risk and Compliance	Privatized statistics that configure CARLA
calibration_audit_log.json	To Risk and Compliance only	Noise per query for the reconstruction and composition checks; never published
Encrypted towns and manifests	From Dataset Generation	Training frames and integrity digests
clean_labels/	From Utility Evaluation	Labels that passed the poisoning gate

File	Direction	Purpose
audit_report.json and audit_certificate.json	To Risk and Compliance	Signed evidence for each run
summary.json	To Utility Evaluation	Attack AUC and test AUC per configuration for the joint figure
privacy_certificate.json	To the release folder	Combined and signed privacy verdict

The AES dataset key and the HMAC signing key traveled separately. Dataset Generation shared the AES key so that the frames could be decrypted. Risk and Compliance received the HMAC key so that it could verify the reports.

5.4 Verification and software testing

The component was tested at three levels. Unit tests checked each module on its own. Examples are the halt at the budget limit, the rejection of a changed report and the regression test that fails if an old key appears in the source. All 107 unit tests passed. Shared library and pipeline tests checked the JSON contracts between the components, and an end-to-end test ran the whole pipeline. GitHub Actions ran the tests of every component and the end-to-end test on each push.

The functional requirements were then checked with the full experiment, which trained twelve models and attacked each one. Table 5.3 lists how each requirement was verified.

Table 5.3: Verification of the functional requirements

ID	Requirement	Verification	Status
FR-01	Runs stay within target ε	ε = 0.991, 3.000 and 7.995	Passed
FR-02	Clipping and noise every step	C = 1.0, σ = 0.977 to 0.507	Passed
FR-03	Training halts at the limit	Halts in a low-limit test	Passed
FR-04	Noise added before simulator	Town01 matched privatized file	Passed
FR-05	Signed report for every run	Altered copy is rejected	Passed
FR-06	Per-step ε is logged	3,090 rows per run	Passed

Repeating a run with a fixed seed gave the same loss and the same test AUC, which confirmed that the runs were reproducible.

5.5 Deployment and operation

The component is installed as a Python package and run stage by stage from the command line. In the intended deployment, an operator first runs the calibration stage on the data owner's machine. Only calibration_stats.json leaves that machine for the simulator. The audit log goes to the auditor through a separate channel. The training and attack stages then run on a GPU machine with access to the encrypted dataset, the clean labels and the AES key. Finally, the certificate stage collects the results of all runs and signs the combined verdict.

The keys are supplied through environment variables or a local .env file, so the same code runs on any machine without changes. Each run writes to its own folder, which makes it safe to repeat an experiment or add a new seed. The budget log, the signed report and its digest stay together in that folder, so an auditor can check any run from its folder alone.

6 RESULTS AND VALIDATION

6.1 Research results

Privacy and utility. Table 6.1 gives the mean over the three seeds for each configuration, and Figure 6.1 plots the two main measures.

Table 6.1: Privacy and utility by configuration (mean ± standard deviation over three seeds)

Configuration	Target ε	Test AUC	Attack AUC	Attack success	Loss gap
No DP	none	0.829 ± 0.009	0.639 ± 0.013	56.7%	0.360
Config A	1.0	0.621 ± 0.045	0.504 ± 0.002	50.3%	0.001
Config B	3.0	0.655 ± 0.035	0.515 ± 0.002	51.6%	0.013
Config C	8.0	0.668 ± 0.020	0.506 ± 0.007	50.8%	0.026

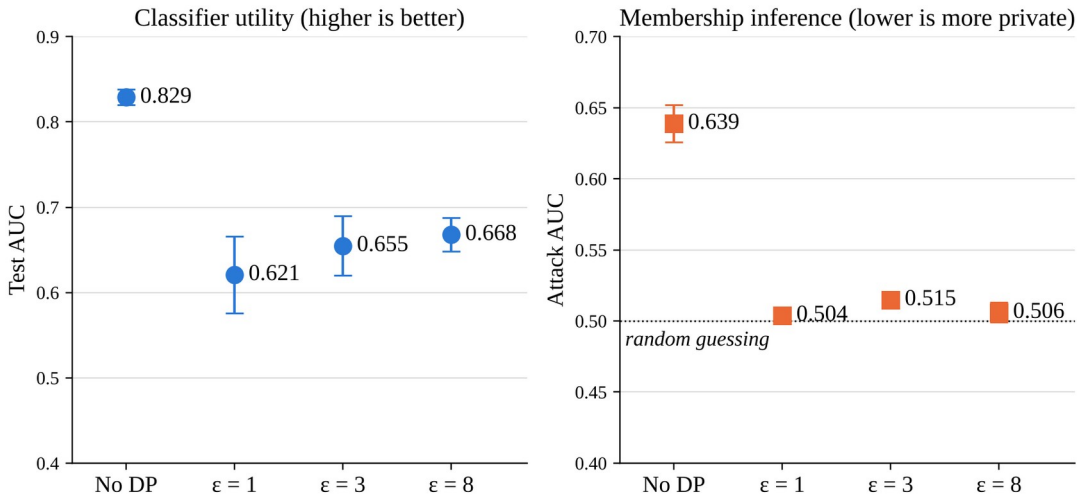


Figure 6.1: Test AUC and attack AUC by configuration (mean and one standard deviation over three seeds)

The attack worked against the non-private baseline, with a mean AUC of 0.639 and a success rate of 56.7%. The baseline loss on training images was 0.36 lower than on unseen images, and this overfitting signal is what the attack uses. Against every private configuration, the attack AUC fell to between 0.504 and 0.515 and the success rate to between 50.3% and 51.6%, close to random guessing. The ranges of single runs did not overlap. The nine private runs had an attack AUC between 0.499 and 0.516, and the three baseline runs between 0.625 and 0.650. The cost was a fall in test AUC from 0.829 to between 0.621 and 0.668.

Paired t-tests against the baseline trained with the same seed showed that both drops were significant at the 0.05 level for every private configuration. Table 6.2 gives the details. With only three seeds, the differences between the three private configurations were not significant.

Table 6.2: Paired t-tests of each private configuration against the baseline (three seeds)

Configuration	Drop in test AUC	p-value	Drop in attack AUC	p-value
Config A	0.208	0.0099	0.134	0.0025
Config B	0.174	0.0084	0.124	0.0032
Config C	0.160	0.0031	0.132	0.0031

Loss-threshold attack. Every saved model was also tested with a loss-threshold attack [19]. It used the same 1,179 members and 1,179 non-members, and every run reproduced the test AUC recorded at training, which confirmed that the splits matched. Following Carlini et al. [6], the true-positive rate (TPR) at a low false-positive rate (FPR) is reported together with the AUC in Table 6.3. Figure 6.2 compares the two attacks.

Table 6.3: Loss-threshold attack (mean ± standard deviation over three seeds; random guessing gives an AUC of 0.5 and a TPR equal to the FPR)

Configuration	Attack AUC	TPR at 1% FPR	TPR at 5% FPR
No DP	0.610 ± 0.026	0.007 ± 0.007	0.057 ± 0.012
Config A	0.501 ± 0.007	0.015 ± 0.002	0.053 ± 0.011
Config B	0.503 ± 0.012	0.016 ± 0.003	0.055 ± 0.013
Config C	0.506 ± 0.012	0.014 ± 0.006	0.053 ± 0.009

The second attack confirmed the main result. Its AUC against the baseline was 0.610 and fell to between 0.491 and 0.518 for every private run. The paired drop of 0.10 to 0.11 was significant for each private configuration ($p = 0.025$, 0.033 and 0.035). At low false-positive rates, however, the attack performed at chance level for every configuration, including the baseline. These values are approximate, because a 1% FPR allows only about 12 false alarms. They still show that the leakage of the baseline

was a small average shift in loss, spread across many images. No single frame could be identified with high confidence, even without DP.

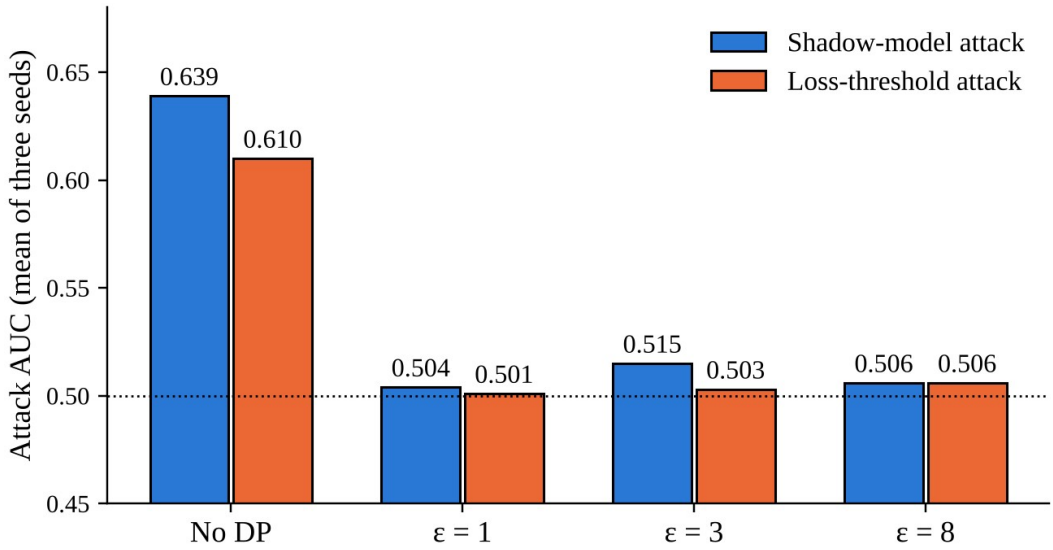


Figure 6.2: Attack AUC of the shadow-model and loss-threshold attacks

Calibration. The calibration profile in the Town01 configuration matched the privatized file field by field, so the simulator used the privatized values. Table 6.4 shows the privatized statistics that configured the simulator. The source values and the noise drawn are kept only in the audit log.

Table 6.4: Privatized calibration statistics sent to Dataset Generation

Statistic	Privatized value
Vehicle density per km	2.5139
Pedestrian density per km	0.9153
Average vehicle speed (km/h)	22.0135
Average pedestrian speed (km/h)	0.2789
Intersections per km	0.2796
Weather: clear, cloudy, rain, fog	0.624, 0.135, 0.241, 0.000
Time of day: morning, afternoon, evening, night	0.144, 0.115, 0.575, 0.167
ε per query, number of queries, total	1.8, 7, 12.6

6.2 Product and technical results

The component was delivered as a working package that runs the whole privacy workflow from calibration to the signed certificate. All six functional requirements passed (Table 5.3). Every private run stayed within its target, with final values of ε = 0.991, 3.000 and 7.995 for Config A, B and C. No run reached the hard limit of 10, and the halt itself was confirmed in a separate low-limit test. Each run logged 3,090 rows of cumulative ε, one for each training step.

The audit trail worked as designed. Every run produced a signed report, and a changed copy of a report was rejected. The final privacy certificate combined the twelve runs, the paired t-tests and the known limitations in one signed file. The package also met its non-functional requirements. No key remained in the source code, and runs were reproducible with a fixed seed. The whole experiment ran on a laptop GPU with 4 GB of memory.

6.3 User or stakeholder validation

The component had no external end users during the project, so validation came from three groups that used or reviewed its outputs.

Supervisors. The supervisor and co-supervisor reviewed the work at the end of each sprint. Their early advice narrowed the privacy work to two clear points, calibration and training. Both could be built and tested within the project timeline.

Risk and Compliance as the user of the evidence. Risk and Compliance used the signed reports, budget logs and calibration audit log as its inputs. It could verify all of them without help from the author, which met the requirement that another party can check every claim. Its composition check also found an error. Earlier versions of the code reported the calibration budget as 1.8, which is the value for each query. The correct sequential total was $7 \times 1.8 = 12.6$. The noise itself was always correct, so the corrected code produced identical statistics and no capture or training had to be repeated. The reports now state the total.

Progress review panel. At the second progress review, the panel commented that even with differential privacy, the reduction in attacks was not significant. This comment is partly correct, and it changed how the results are presented in this thesis. In statistical

terms, the drop is significant (Table 6.2). In absolute terms, however, it is small. The baseline itself leaked only moderately, with a success rate of 56.7%, so DP had little leakage to remove. At a 1% FPR, the baseline was no better than chance. In response, Chapter 7 now separates statistical significance from the size of the effect. A stress test against a more strongly overfitted baseline is listed as the first item of future work (Section 9.4).

6.4 Integration results

The integration was checked from both sides of each interface. Dataset Generation configured the simulator from calibration_stats.json, and the Town01 profile matched the privatized file field by field. Utility Evaluation used summary.json to draw the joint privacy and utility figures of the group. Risk and Compliance re-ran four checks on the outputs of the component with its own code. Table 6.5 lists the results.

Table 6.5: Independent verification of the component by Risk and Compliance

Check	What was tested	Result	Outcome
Attack 1	All signed reports pass verification	4 of 4 reports verified; signatures matched	Passed
Attack 1	Tampering with a report is detected	Changing ε from 0.990572 to 0.01 was caught	Passed
Attack 2	Reported ε matches the training log	All 3 configurations within 0.0001	Passed
Attack 2	ε never exceeds the ceiling of 10	Peaks of 0.99, 3.00 and 7.99	Passed
Attack 3	Calibration noise ratio near 1.0	Ratio of 0.94, inside 0.5 to 1.5	Passed
Attack 4	MIA fails against DP models	AUC between 0.50 and 0.53 for all DP models	Passed
Attack 4	Results match those of Privacy Integration	Within about 2% for all DP configurations	Passed

The budget audit found that the reported final ε of every private configuration was identical to the last value in its training log. The values were 0.990572, 2.99984 and 7.995129 against targets of 1, 3 and 8, and ε never went down from one step to the next. Figure 6.3 compares these values with the targets and the limit.

The reconstruction check compared the published calibration values with the source values. Across the five scalar fields, the mean absolute difference was 0.6246, against an expected value of 0.6667 for a correct Laplace mechanism. The ratio of 0.94 is close to the ideal of 1.0.

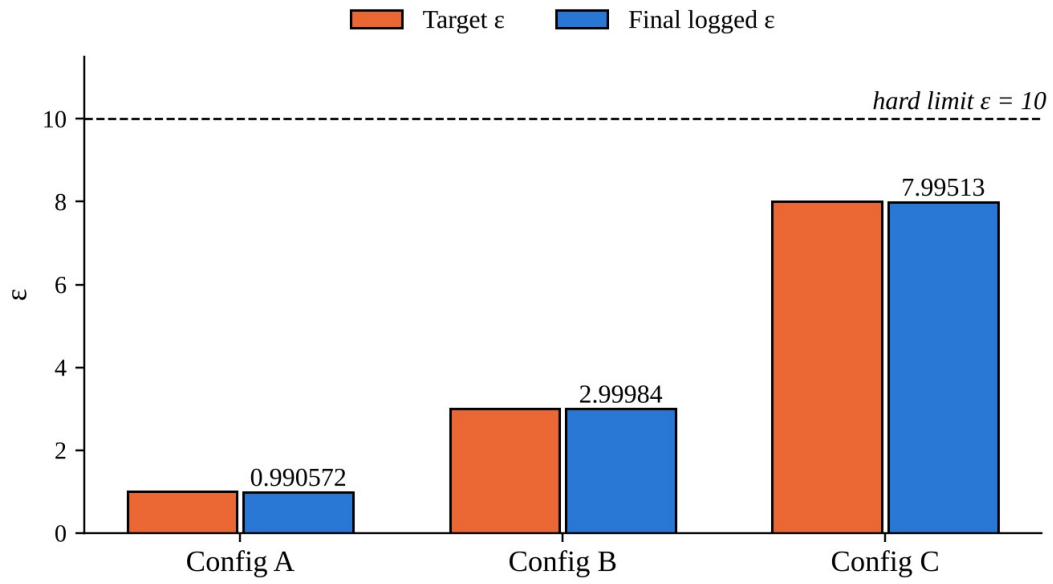


Figure 6.3: Target and final logged ε of the three private configurations (independent audit by Risk and Compliance)

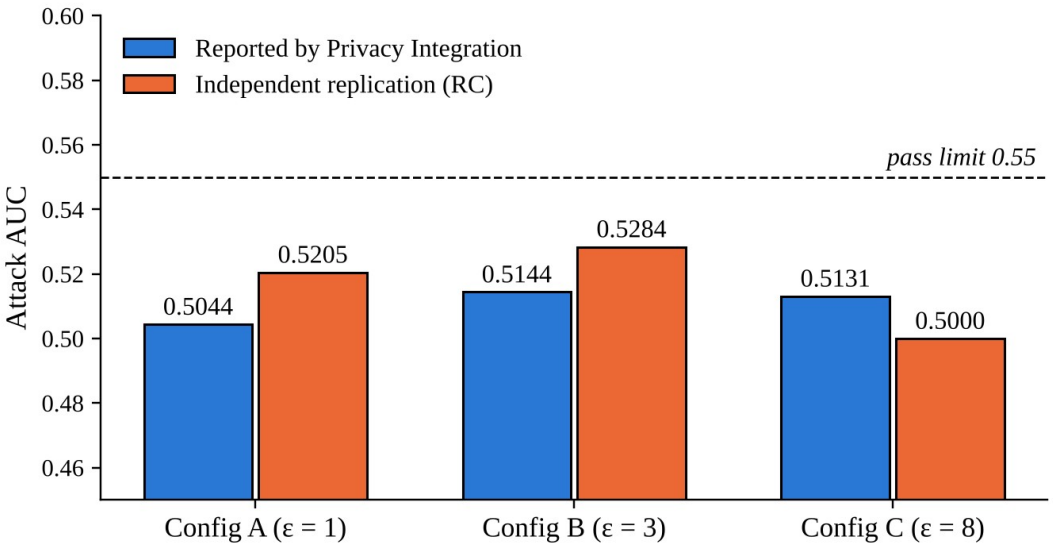


Figure 6.4: Attack AUC reported by Privacy Integration and found by the independent replication

The independent membership inference attack gave AUC values of 0.5205, 0.5284 and 0.5000 for Config A, B and C. The signed audit reports of the component, one for each

configuration, had recorded 0.5044, 0.5144 and 0.5131. The largest difference is about 2%, which is reasonable for two separately written attacks on the same data. Against the non-private baseline, the independent attack gave an AUC of 0.65, close to the mean of 0.639 reported here. Figure 6.4 compares the two sets of results.

6.5 Summary of findings

- The Laplace mechanism privatized all seven calibration statistics, and the simulator used only the privatized values. The total calibration budget was 12.6, above the planned limit of 2.0.
- Every private training run stayed within its target ε and far below the limit of 10.
- The shadow-model attack succeeded against the baseline (AUC 0.639, success 56.7%) and fell to near chance against every private model (AUC 0.504 to 0.515, success 50.3% to 51.6%). The drop was significant for all three budgets.
- The loss-threshold attack confirmed the result. At a 1% FPR, no model leaked more than chance, including the baseline.
- The cost of privacy was a drop of 0.16 to 0.21 in test AUC, with no significant difference between the three budgets.
- Risk and Compliance verified the reports, the budget logs and the attack results on its own. Its check also found a reporting error in the calibration total, which was corrected.

7 DISCUSSION

7.1 Interpretation of findings

The results answer the research question with a yes, with some limits. For RQ1, the calibration step privatized all seven statistics, and the simulator was used in the same way as before. For RQ2, DP-SGD removed the membership signal that the attacks used. The same attack succeeded against the baseline, so a broken attack cannot explain the results near 50% on the private models. For RQ3, the protection cost a measurable drop of 0.16 to 0.21 in test AUC.

The size of the effect needs a careful reading. The drop in attack AUC, from 0.639 to about 0.50, is statistically significant, but it starts from a baseline that leaked only moderately. A success rate of 56.7% means the attack was right a little more often than a coin toss. At a 1% FPR, the loss-threshold attack could not pick out any frame with confidence, even on the baseline. The leakage that DP removed was therefore a small average shift in loss, spread across many images. The review panel raised this point (Section 6.3). DP-SGD brought a moderate leak down to chance level. Since no severe leak was present, the results cannot show how DP-SGD would handle one.

The role of ε also matters. The three budgets gave almost the same result against the attacks, even though $\varepsilon = 8$ is a much weaker formal bound than $\varepsilon = 1$. Most of the utility cost came from using DP-SGD at all, and the choice of budget added comparatively little. For this model and data, a looser budget kept almost all of the measured protection at a slightly lower cost.

7.2 Comparison with literature baselines and existing solutions

The gap between the formal bound and the measured leakage matches earlier work. Jayaraman and Evans [10] found that measured leakage at a given ε varies widely and is often far below the bound. Nasr et al. [11] showed that a worst-case attacker can reach the bound, while realistic attackers fall well short of it. The attacks used here are realistic attacks, so it is expected that $\varepsilon = 1$, 3 and 8 all gave results near chance.

The low-FPR result agrees with Carlini et al. [6]. An AUC of 0.639 looks like clear leakage, but the TPR at a 1% FPR showed that no single record was exposed with confidence. With both measures in the report, the leak of the baseline was neither

hidden nor overstated. The overfitting signal also behaved as Yeom et al. [19] described. The baseline had a loss gap of 0.36, and the private models had gaps of 0.001 to 0.026, which matches the fall in attack success.

The component also fills gaps left by existing tools. Simulators such as CARLA [4] leave the calibration step unprotected, while here it receives a formal guarantee. Training libraries such as Opacus [20] apply DP-SGD but do not test the trained model. Auditing tools such as ML Privacy Meter [21] measure leakage but do not apply privacy or link the result to a signed budget record. The component joins these steps into one workflow whose output another party can check.

7.3 Research contribution

The main research contribution is a measured link between the formal budget ε and attack resistance for models trained on synthetic driving images. The study tested three budgets with three seeds each, against a positive control and two different attacks. All three budgets reduced a moderate leak to chance level. In this setting, the choice of budget had little effect on either privacy or utility.

A second contribution is a method for reporting such results. It reports the AUC, the success rate and the TPR at low FPR together, alongside the loss gap and paired tests against same-seed baselines. A third contribution is the finding that the calibration channel is a privacy point in its own right. Seven queries at $\varepsilon = 1.8$ each add up to $\varepsilon = 12.6$ under sequential composition. This shows that a per-query budget can look small while the total is not.

7.4 Product contribution

The product contribution is the privacy_integration package. It privatizes calibration statistics, trains models under a hard budget limit, attacks them and signs the results. Its outputs can be checked without rerunning any training. During the project, the independent check by Risk and Compliance used these outputs to find a reporting error that the author had missed. Without the signed logs and the separate check, the error in the calibration total would have gone unnoticed.

7.5 Limitations and threats to validity

Internal validity. Three seeds per configuration were enough to separate the private models from the baseline, but not to separate the three budgets from each other. Opacus secure mode was turned off, because the torchcsprng library it needs is no longer maintained and does not support PyTorch 2.x. The noise therefore came from a standard random number generator.

Construct validity. Privacy was measured with a binary presence classifier, not with the YOLOv8 detector that Utility Evaluation used. The attacks used here are realistic but not the strongest known. A likelihood ratio attack [6] might find leakage that these attacks missed. The baseline also leaked only moderately, so the study shows that DP removes a moderate leak, but not how it handles a severe one.

External validity. The calibration source was reference statistics from CARLA runs, not a real dataset. The total calibration budget of 12.6 is above the planned limit of 2.0. The results come from one simulator, one model and one task, so they may not hold for larger models or other kinds of data.

Utility. Utility Evaluation trained the same detector on synthetic data and on real KITTI images. The synthetic model reached 10.46% of the real-data mAP50. The proposal target of staying within 15% of the real-data baseline was therefore not reached with the current synthetic data. That detector was trained without DP-SGD, so the gap does not come from the training step of this component. The dataset was generated at one calibration budget only, so the share of the gap caused by calibration noise could not be measured.

7.6 Implications

For practitioners, the results suggest that a formal ε is not enough evidence on its own. A team should attack its own models, include a non-private control and report the low-FPR result as well as the AUC. For small models like the one used here, a moderate budget such as $\varepsilon = 8$ may give nearly the same measured protection as $\varepsilon = 1$ at a lower cost. The formal guarantee is still much weaker, so this should be judged case by case.

For the design of synthetic data pipelines, the calibration channel needs the same care as training. Budgets must be added up over all queries that read the same records, and

the noise log must be treated as sensitive. For auditors and regulators, signed reports and per-step budget logs show that privacy evidence can be checked without trusting the team that produced it.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The component gives a team that builds perception models with simulated data a privacy record it can show to others. It turns a formal budget into evidence: a signed budget log, attack results against private and non-private models, and a certificate that another party can verify. With this record, a team can use real driving statistics to make its simulation realistic. It can still show that those statistics, and the models trained later, were protected.

The project proposal identified four groups of users:

- **Autonomous vehicle developers**, mainly machine learning engineers and data infrastructure leads, who need training data at scale without long privacy reviews of every dataset.
- **Data protection officers and regulators**, who need evidence they can read and check.
- **Academic researchers**, who want to share synthetic datasets with a clear privacy record.
- **Road users**, whose faces, plates and routes should not be exposed through the training pipeline.

8.2 Market and competitive position

The target market is teams that already use simulation for perception training. Examples are autonomous vehicle start-ups, automotive suppliers, university research labs and simulation service providers. Many of them want to use real statistics to make simulations realistic, but privacy rules and contracts prevent them from sharing raw recordings.

The competing approaches each cover only part of the need. Face and plate blurring removes visible identifiers but does not protect the trained model. Simulators such as CARLA and NVIDIA DRIVE Sim produce realistic data, but the project proposal found no formal privacy guarantee for their calibration step. Training libraries such as Opacus apply DP-SGD but do not test the result. Auditing tools such as ML Privacy Meter test models but do not apply privacy. This component joins these steps. It

protects the calibration channel, applies DP-SGD, attacks the result with a positive control and signs the evidence.

8.3 Business or sustainability model

The most realistic model is an open-source core with paid services. The privacy_integration package could be released as an open-source toolkit, which lowers the barrier for research labs and builds trust in the method. Companies could pay for integration with their own simulators and training pipelines, for support and for training. A hosted service could also run attacks and produce signed privacy reports for models that a customer uploads. Academic licenses or free use for research would help the method spread and keep it under review by others.

The running costs are low. The whole experiment in this thesis ran on a laptop GPU with 4 GB of memory, and the main dependencies are free, open-source libraries. The largest cost would be the staff time needed to adapt the component to each customer's data and models.

8.4 Scalability intellectual property and compliance

Scalability. The calibration step is cheap, because it adds noise to a few statistics. The training and attack steps grow with the size of the model and data. Larger detectors would need more GPU memory, because DP-SGD computes a gradient for every sample. Opacus supports larger models, but each new model would need its own tuning under DP-SGD. This study needed such tuning for the optimizer and the normalization layer.

Intellectual property. The component builds on open-source libraries such as PyTorch, Opacus and CARLA. Any release must respect their licenses and keep their notices. The code written for this component belongs to the author and the project, subject to the rules of the university. KITTI was used by the group only as a utility baseline under its non-commercial license, so a commercial product could not use it in the same way.

Compliance. The component produces evidence that a data controller can use for compliance. Its evidence maps to GDPR Articles 5(1)(c), 5(2) and 25 [3] and to the

NIST Privacy Framework [23]. The final legal decision stays with the data controller. Any product must also state its limits clearly, such as the calibration total of 12.6 and the missing secure noise source. Otherwise, users may trust the guarantee more than they should.

8.5 Contribution to the Sustainable Development Goals

The component contributes most directly to two Sustainable Development Goals.

- **SDG 9: Industry, Innovation and Infrastructure.** Target 9.5 asks for stronger research and technological capability. The component gives developers a tested way to use synthetic data with a formal privacy record, which supports privacy-preserving innovation in the automotive sector.
- **SDG 16: Peace, Justice and Strong Institutions.** Target 16.10 includes the protection of fundamental freedoms, and privacy is one of them. The component helps keep personal data out of training pipelines and gives regulators evidence they can verify.

The link to road safety, under SDG 3 (target 3.6) and SDG 11 (target 11.2), is indirect. Better access to safe training data may help perception models improve, but this thesis did not measure any effect on safety. There are also possible negative effects. A signed certificate could give a false sense of security if users read ε without the attack results and limitations. Training many models also uses energy. These risks can be reduced by always publishing the attack results and limitations with the certificate, and by keeping models small where that is enough.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research made the privacy of synthetic autonomous vehicle data measurable, and checkable by others. The Privacy Integration component applies the Laplace mechanism to the statistics that configure CARLA. It applies Opacus DP-SGD with an RDP accountant to the models trained on the generated frames. It then attacks its own models and signs the results.

Over three seeds, the shadow-model attack reached a success rate of 56.7% against the unprotected model. Against every private model, it reached only 50.3% to 51.6%, at a cost of 0.16 to 0.21 in test AUC. A second, loss-threshold attack gave the same result. Neither attack could identify individual frames with high confidence for any model. The main finding is therefore that DP-SGD reduced a moderate leak to chance level, and that the three budgets gave almost the same protection. The signed reports also let Risk and Compliance check the evidence on its own. That check found an error in the reported calibration total, which was then corrected.

9.2 Achievement of objectives

Table 9.1 compares the results with the objectives in Section 1.5.

Table 9.1: Achievement of objectives

Objective	Outcome	Status
Select a DP training library	Opacus 1.4.1 with per-sample gradients and RDP accounting	Achieved
Calibration budget below $\varepsilon = 2$	Laplace noise on all seven queries; sequential total of 12.6	Partially achieved
DP-SGD with an automatic limit	All runs within target ε (0.991 to 7.995) and the limit of 10	Achieved
Attack success at or below 55%	50.3% to 51.6% with DP, 56.7% without	Achieved
Privacy and utility over budgets and seeds	Three budgets, three seeds each, paired t-tests	Achieved
Signed evidence verified by another party	All reports and logs verified by Risk and Compliance	Achieved
Utility within 15% of real data (group target)	10.46% of the real-data mAP50 (Utility Evaluation)	Not reached

9.3 Individual research and product contributions

The individual research contributions of this thesis are as follows:

- A measured test of how the formal budget ε relates to membership inference on synthetic driving images, with a positive control and two attacks.
- Evidence that, in this setting, ε = 1, 3 and 8 all reduce a moderate leak to chance level at a similar utility cost.
- A reporting method that combines AUC, success rate, TPR at low FPR, loss gap and paired tests.
- Evidence that the budgets of all calibration queries must be added up. Seven queries at ε = 1.8 each give a total of 12.6.

The individual product contributions are as follows:

- The privacy_integration package with its six modules.
- The DP training wrapper with a hard budget limit.
- The classifier and label rule that made private training work on the CARLA frames.
- The signed audit reports and the privacy certificate.
- The JSON contracts and 107 unit tests that connect the component to the rest of the framework.

9.4 Future directions

The future work is listed in order of priority.

1. **Stress test against a stronger leak.** Train a baseline that overfits more strongly, for example on a smaller training set for more epochs. Then repeat the private configurations under the same conditions. This would show whether DP-SGD also removes a severe leak, which answers the main concern of the review panel.
2. **Stronger attacks.** Add the likelihood ratio attack of Carlini et al. [6] and report the TPR at 0.1% FPR.
3. **Stricter calibration budget.** Run each query at ε = 1.8 / 7 ≈ 0.257, so that the total stays near 1.8, and take the source statistics from a real dataset.

4. **More seeds.** Use more seeds per configuration, so that the three budgets can be separated statistically.
5. **Secure noise.** Use a cryptographically secure random number generator that works with current PyTorch versions.
6. **Public-key signatures.** Replace HMAC with public-key signatures, so that any third party can verify the reports without a shared key.

REFERENCES

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354-3361.

[2] P. Sun *et al.*, "Scalability in perception for autonomous driving: Waymo Open Dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2446-2454.

[3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," *Off. J. Eur. Union*, vol. L 119, pp. 1-88, May 2016.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 78, 2017, pp. 1-16.

[5] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2017, pp. 3-18.

[6] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 1897-1914.

[7] T. Stadler, B. Oprisanu, and C. Troncoso, "Synthetic data: Anonymisation groundhog day," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1451-1468.

[8] L. Xie, K. Lin, S. Wang, F. Wang, and J. Zhou, "Differentially private generative adversarial network," 2018, *arXiv:1802.06739*.

[9] R. Torkzadehmahani, P. Kairouz, and B. Paten, "DP-CGAN: Differentially private synthetic data and label generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2019, pp. 98-104.

[10] B. Jayaraman and D. Evans, "Evaluating differentially private machine learning in practice," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1895-1912.

[11] M. Nasr, S. Song, A. Thakurta, N. Papernot, and N. Carlini, "Adversary instantiation: Lower bounds for differentially private machine learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2021, pp. 866-882.

[12] M. Abadi *et al.*, "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 308-318.

[13] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. 3rd Theory Cryptogr. Conf. (TCC)*, ser. Lecture Notes Comput. Sci., vol. 3876. Berlin, Germany: Springer, 2006, pp. 265-284.

[14] C. Dwork, "Differential privacy," in *Proc. 33rd Int. Colloq. Automata, Lang. Program. (ICALP)*, ser. Lecture Notes Comput. Sci., vol. 4052. Berlin, Germany: Springer, 2006, pp. 1-12.

[15] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3-4, pp. 211-407, 2014.

[16] I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Comput. Secur. Found. Symp. (CSF)*, 2017, pp. 263-275.

[17] C. Xie, Z. Cao, Y. Long, D. Yang, D. Zhao, and B. Li, "Privacy of autonomous vehicles: Risks, protection methods, and future directions," 2022, *arXiv:2209.04022*.

[18] G. Ganev, "When synthetic data met regulation," 2023, *arXiv:2307.00359*.

[19] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *Proc. IEEE 31st Comput. Secur. Found. Symp. (CSF)*, 2018, pp. 268-282.

[20] A. Yousefpour *et al.*, "Opacus: User-friendly differential privacy library in PyTorch," 2021, *arXiv:2109.12298*.

[21] S. K. Murakonda and R. Shokri, "ML Privacy Meter: Aiding regulatory compliance by quantifying the privacy risks of machine learning," 2020, *arXiv:2007.09339*.

[22] Y. Wu and K. He, "Group normalization," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 3-19.

[23] Nat. Inst. Standards Technol., "NIST Privacy Framework: A tool for improving privacy through enterprise risk management, Version 1.0," U.S. Dept. Commerce, Gaithersburg, MD, USA, 2020.

[24] Nat. Inst. Standards Technol., "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," U.S. Dept. Commerce, Gaithersburg, MD, USA, Rep. NIST AI 100-1, 2023.

[25] *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2015.

GLOSSARY

Differential privacy. A property of a randomized computation that limits how much any single record can change its output.

Privacy budget (ε). The number that sets the strength of differential privacy. A smaller ε means stronger protection and more noise.

Failure probability (δ). The small probability with which the ε limit of (ε, δ)-differential privacy may fail.

Sensitivity (Δf). The largest change that one record can cause in the answer to a query.

Sequential composition. The rule that the budgets of several private queries on the same records add up.

Post-processing. Any further computation on a private output, which cannot weaken its guarantee.

Noise multiplier (σ). The ratio of the standard deviation of the DP-SGD noise to the clipping norm C.

Privacy accountant. A component that tracks the cumulative ε spent during training.

Membership inference. An attack that tries to tell whether a given record was in the training set of a model.

Shadow model. A model trained by an attacker to copy the behavior of a target model.

Positive control. A case in which the attack is expected to succeed, used to show that the attack works.

Loss gap. The difference between the mean loss of unseen records and that of training records.

APPENDICES

Appendix A Extended results

This appendix gives the signed evidence and the calibration audit in more detail than Chapter 6. Table A.1 lists the privacy record of each private configuration, as found in the signed audit reports and the independent audit.

Table A.1: Privacy record of each private configuration

Run	Target ε	σ	Final logged ε	Steps logged	Attack AUC in report	Independent attack AUC
Config A	1.0	0.977	0.990572	3,090	0.5044	0.5205
Config B	3.0	0.665	2.99984	3,090	0.5144	0.5284
Config C	8.0	0.507	7.995129	3,090	0.5131	0.5000

Each final logged ε is slightly below its target. Opacus chooses σ so that the budget at the final step does not pass the target. In the tamper test, the recorded ε in a copy of the Config A report was changed from 0.990572 to 0.01. Verification failed at once.

Table A.2 summarizes the reconstruction check on the calibration channel, carried out by Risk and Compliance with the private audit log.

Table A.2: Reconstruction check on the calibration channel

Measure	Value
Mean absolute difference between published and source values (five scalar fields)	0.6246
Expected mean absolute difference for a correct Laplace mechanism	0.6667
Ratio of observed to expected difference	0.94
Largest distortion (intersections per km)	76.8%
Total calibration budget under sequential composition	7 × 1.8 = 12.6

Appendix B Academic integrity and AI use disclosure

Generative AI was used while preparing this thesis, in line with the module policy on AI use.

Tool. Claude, a large language model by Anthropic.

Use. The tool was used to create the figures and diagrams from the author's own results, and to help with small verification checks on the document.

Verification. The author checked the numbers, citations and claims in the thesis against the experiment outputs and the cited sources.

Responsibility. The research design, the experiments, the results and the conclusions are the responsibility of the author.